

fastWorkflow: Closing the Performance Gap Between Small and Frontier Language Models for Conversational Agents

Sanchit Satija
sanchitsatija55@gmail.com
Radiant Logic
San Rafael, California, USA

Priyanshu Jani
anshoo.jani@gmail.com
Radiant Logic
San Rafael, California, USA

Aditya Bhatt
adihbhatt04@gmail.com
Radiant Logic
San Rafael, California, USA

Dhar Rawal
drawal@radiantlogic.com
Radiant Logic
San Rafael, California, USA

Abstract

Large language models are increasingly deployed in conversational agents that assist humans with complex, multi-step tasks, yet their deployment at scale is constrained by high inference costs, latency, and data privacy concerns. Small language models (SLMs) offer compelling operational advantages but exhibit systematic failure modes in agentic settings, particularly in conversational workflows: domains where tasks are solved interactively by a human and an LLM through structured tool invocation. Despite growing SLM deployment, these agentic failure modes remain poorly characterized and inadequately addressed. We present an empirically-grounded taxonomy categorizing SLM failures across five dimensions: natural language understanding failures, tool management failures, task decomposition and sequencing failures, agentic reasoning failures, and context management failures, and quantify their prevalence on the τ -bench benchmark. Guided by this taxonomy, we introduce fastWorkflow, a dual-mode agentic architectural framework implementing a cascaded NLU pipeline for intent detection and structured parameter extraction with validation, hierarchical context organization that reduces effective action space, explicit task planning with dependency-aware decomposition, and adaptive context management, among other targeted mitigations. On τ -bench, GPT-OSS-20B augmented with fastWorkflow achieves 83.47% Pass¹ on the Retail domain and 78% on Airline, surpassing all frontier models evaluated on τ -bench leaderboard including Claude Sonnet 4 (80.5% Retail, 60.0% Airline) and Claude Opus 4.1 (82.4% Retail, 56.0% Airline), while operating at $\sim 22\times$ lower inference cost. Even Mistral-7B-Instruct with fastWorkflow matches Claude Sonnet 4 on Airline at 60%. Ablation studies confirm that the cascaded NLU pipeline is the most impactful component, with its removal causing performance collapses of 58 points on Retail and 68 points on Airline. Our findings demonstrate that architectural separation of concerns, offloading error-prone operations to structured subsystems while preserving LLM flexibility for planning and recovery, can close the performance gap between small and frontier models in conversational workflow tasks, shifting the cost-performance

Pareto frontier for production deployment in domains involving multi-turn, tool-augmented human-LLM collaboration.

CCS Concepts

• Computing methodologies → Artificial intelligence.

Keywords

Small Language Models, Conversational Agents, Natural Language Understanding, Tool Use, Failure Taxonomy

ACM Reference Format:

Sanchit Satija, Aditya Bhatt, Priyanshu Jani, and Dhar Rawal. 2026. fastWorkflow: Closing the Performance Gap Between Small and Frontier Language Models for Conversational Agents. In *ACM Conference on AI and Agentic Systems (ACM CAIS '26)*, May 26–29, 2026, San Jose, CA, USA. ACM, New York, NY, USA, 20 pages. <https://doi.org/10.1145/3786335.3813158>

1 Introduction

Large language models have emerged as powerful foundations for building conversational agents capable of executing complex, multi-step tasks through iterative tool use [25, 36]. These *agentic systems* extend beyond single-turn question answering to maintain state across interactions, invoke external tools [27], and recover from errors which require exhibiting capabilities essential for real-world applications such as customer service automation, software engineering agents, and enterprise workflow management.

In addition to cost, frontier models introduce significant challenges in latency, throughput, and data privacy. Large models typically exhibit higher end-to-end response times due to network overhead, model size, and queueing delays under load, which can be prohibitive for real-time or user-facing applications. Agentic systems aggravate this issue, as sequential calls compound latency across multiple inference steps. Moreover, routing sensitive user data through third-party inference endpoints raises significant privacy concerns. In contrast, small language models (SLMs) can be deployed locally or on dedicated infrastructure [19], keeping data entirely within organizational boundaries while enabling lower and more predictable latency, higher throughput, and tighter integration with existing systems.

However, small models exhibit systematic failure modes in agentic settings that do not manifest in standard benchmarks. While an



This work is licensed under a Creative Commons Attribution 4.0 International License. *ACM CAIS '26*, San Jose, CA, USA

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2415-2/26/05

<https://doi.org/10.1145/3786335.3813158>

⁰Code/Artifacts and Full Paper with Appendix:

<https://github.com/radiantlogicinc/fastworkflow>

SLM may achieve competitive performance on MMLU, HumanEval and some other benchmarks [7, 20, 31], the same model fails catastrophically when errors compound across multi-turn interactions [38], when tool schemas demand precise argument formatting, or when recovery from mistakes requires meta-cognitive reasoning about prior failures [10]. These *agentic failure modes* represent a distinct challenge from general capability limitations: they arise from the interaction between model capacity and the specific demands of iterative, stateful computation.

Despite growing deployment of SLM-based agents [3], the research community lacks a systematic characterization of these failure modes and principled approaches to mitigate them. The intersection, i.e., understanding *in what ways* small models fail in conversational workflow settings and *how* to architect systems that compensate for these failures remains underexplored.

This paper addresses this gap through three contributions:

- (1) **A Taxonomy of Agentic Failure Modes.** We present an empirically-grounded taxonomy categorizing failures across five dimensions: *intent detection*, *parameter extraction*, *task decomposition and sequencing*, *agentic reasoning*, and *context management*. We quantify the prevalence of each failure type in multi-turn, tool-augmented conversational settings.
- (2) **Introducing *fastWorkflow*.** We introduce *fastWorkflow*, a dual-mode framework where a workflow agent operates with the aid of an assistant. The assistant implements a cascaded NLU pipeline for intent detection, structured and iterative parameter extraction with parameter validation, and tool organization into context hierarchies that mirror application structure. The main agent is equipped with a task planner that generates step-by-step execution plans, incorporating trajectory-informed replanning as well as adaptive context management. The assistant provides proper command execution scaffolding, error recovery mechanisms, and fallback strategies when agentic reasoning fails.
- (3) **Comprehensive Results and Ablation Study.** We demonstrate on τ -bench [34] that small open-source models augmented with *fastWorkflow* achieve better performance than frontier models while maintaining substantial cost advantage. We conduct ablations isolating the contribution of each component, identifying which mitigations matter most.

2 Related Work

LLM-Based Agents, Benchmarks, and Failure Analysis. The ReAct framework [36] established interleaving chain-of-thought reasoning with environment actions, later augmented by Reflexion [28] with verbal self-reflection. Several works demonstrate small models can match frontier agents: TinyAgent [5] targets on-device function calling, Agent Lumos [37] decomposes reasoning into modular stages, and effGen [29] optimizes SLM agents, outperforming LangChain, AutoGen, and Smolagents across 13 benchmarks. The NVIDIA Nemotron-Orchestrator 8B [30] demonstrates efficient tool orchestration on TAU2. On the evaluation side, AgentBench [18] reveals large gaps between commercial and open-source models, τ -bench [34] evaluates multi-turn customer service interactions, and MAST [22] provides a failure taxonomy across seven multi-agent frameworks. Our taxonomy (Section 3) complements these by

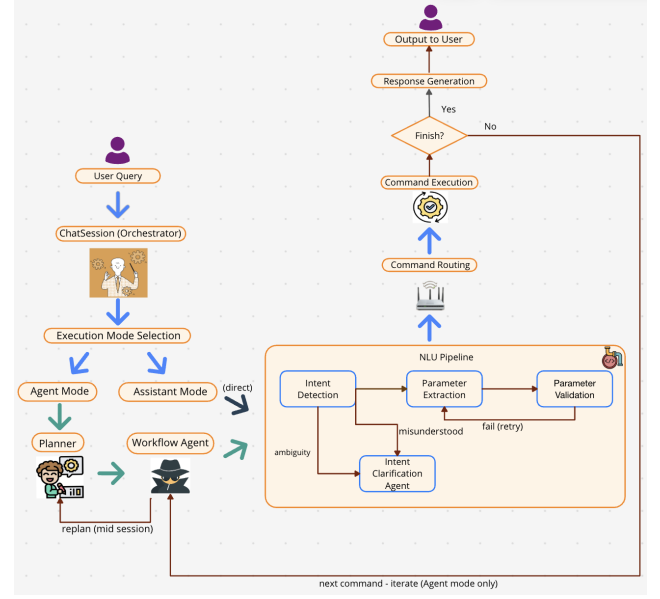


Figure 1: High-level overview of the *fastWorkflow* architecture. This design effectively decouples error-prone natural language understanding tasks from multi-step agentic reasoning.

characterizing failures specific to small models and their cascading interactions.

NLU, Tool Use, and Task Planning. Classical task-oriented dialogue decomposes understanding into intent detection and slot filling [24]; FnCTOD [16] reframes this as function calling. For tool use, Toolformer [27] introduces self-supervised tool learning, Gorilla [23] reduces API hallucination via retriever-aware fine-tuning, and ToolLLM [26] scales to 16,464 APIs. For planning, Tree of Thoughts [35] and Plan-and-Solve [32] prompt explicit task decomposition, while He et al. [9] formalize LLM planning as POMDP navigation and RAP [8] applies MCTS with the LLM as world model. *fastWorkflow* addresses all three concerns through a cascaded NLU pipeline, context hierarchies, and dependency-graph navigation.

Context Management and Knowledge Transfer. Long contexts degrade LLM performance when relevant information appears in the middle, exhibiting a U-shaped primacy-recency bias [17]. LLMingua [12] addresses this via token-level prompt compression. For knowledge transfer, ExpeL [39] and AutoGuide [6] extract guidelines from agent experiences via single-model self-assessment, while Agent Distillation [13] transfers behavior into 0.5B–3B models via fine-tuning on teacher trajectories.

3 Failure Mode Taxonomy

We model agentic task completion as a stateful interaction where an agent invokes commands $f \in F$ with parameters θ to satisfy a user goal G , formalized as a POMDP in Appendix A.1. Below we present an empirically-grounded taxonomy of failure modes exhibited by small language models in conversational workflow settings: domains where a human and an LLM collaborate interactively

through structured tool invocation, with the human supplying high-level reasoning and feedback while the system handles command execution. These failures are categorized along five dimensions: *Natural Language Understanding* (intent detection and parameter extraction), *Tool Management*, *Task Decomposition and Sequencing*, *Agentic Reasoning*, and *Context Management*. Table 1 summarizes the complete taxonomy.

3.1 Natural Language Understanding Failures

NLU failures occur at the interface between user utterances and workflow command invocation. They represent the earliest point of failure in the agent pipeline: if the system cannot correctly identify what the user wants or extract the necessary parameters, no amount of downstream reasoning can recover.

3.1.1 Intent Misclassification. Intent misclassification occurs when the agent maps a user utterance to an incorrect workflow command. We observe four characteristic patterns: *semantic confusion*, where the model conflates commands with overlapping surface-level semantics (e.g., routing “*I want to change my flight*” to cancel reservation command instead of modify reservation command); *context-insensitive routing*, where the model ignores the contextual constraint and attempts to invoke commands outside the active command set $F_c \subseteq F$; *ambiguity collapse*, where genuinely ambiguous utterances that map plausibly to multiple commands are committed to a single interpretation without seeking clarification, a behavior less prevalent in larger models that more frequently hedge or ask disambiguating questions; and *hallucinated command invocation*, where the model attempts to invoke commands that do not exist in F_c altogether.

3.1.2 Parameter Extraction Failures. Even when intent is correctly identified ($\hat{f} = f^*$), the agent must extract parameters θ conforming to the command’s schema Θ_f . We observe six characteristic sub-patterns: *missing required parameters*, where the model fails to extract θ_{req} from conversation history; *type coercion errors*, where integers, enums, or structured JSON, etc are returned as incorrect types; *constraint violations*, where plausible-looking values nonetheless fail declared regex patterns or valid ranges; *hallucinated values*, where the model fabricates parameter values absent from the conversation history, risking operations on incorrect data; *misassigned values*, where a real value is extracted but assigned to the wrong parameter (e.g., an order id populating a customer id field); and *cross-turn parameter fragmentation*, where incremental multi-turn parameter provision is mishandled. Small models struggle to maintain a coherent parameter accumulation across turns.

3.2 Tool Management Failures

Tool management failures arise from the interaction between model capacity and the size and complexity of the available action space. Tool management failures concern the agent’s ability to navigate and reason over the space of available tools.

3.2.1 Tool Overload. When small models are exposed to a large number of tools at once, their performance degrades significantly. Specifically, when the number of available workflow commands $|F_c|$ is large, small models exhibit a marked degradation in selection

accuracy. The model’s capacity to discriminate between tools and context decreases as the size of the available action space grows.

3.2.2 Schema Hallucination. Small models may fabricate argument names and structures that do not conform to any defined schema of the tools. Schema hallucination occurs more frequently when (a) the model has been exposed to different tool schemas during pretraining (causing interference from memorized schemas), or (b) the current task requires a capability not directly provided by any available tool, prompting the model to “invent” one.

3.3 Task Decomposition and Sequencing Failures

Task Decomposition and Sequencing failures occur when the agent cannot construct or execute a viable sequence of actions to accomplish a multi-step goal. These failures are distinct from individual action-level errors; they represent higher-order reasoning deficits about task structure.

3.3.1 Task Decomposition Failure. Multi-step tasks require the agent to decompose a high-level user goal into an ordered sequence of sub-goals. Failure occurs when the agent *under-decomposes* (attempts a complex goal in a single, insufficient command), *over-decomposes* (fragments the task into unnecessarily fine-grained steps, compounding error probability), or *mis-decomposes* (pursues sub-goals irrelevant or inconsistent with the original request).

3.3.2 Dependency Blindness. Even when sub-goals are correctly identified, the agent must respect ordering constraints imposed by data dependencies and preconditions. Dependency blindness occurs when the agent executes sub-goals in an order that violates these constraints. This includes both direct ordering violations as well as parameter dependency failures where the agent fails to recognize that information required by one command can only be obtained by first executing another command. Small models often fail to recognize these data flow dependencies and instead either hallucinate the missing parameter, omit it entirely and trigger an error, or prematurely declare the task impossible without recognizing the necessary prerequisite step.

3.4 Agentic Reasoning Failures

Agentic reasoning failures encompass deficits in the agent’s ability to monitor its own progress, adapt to feedback, and make meta-level decisions about task execution. These failures are uniquely agentic, they do not manifest in single-turn evaluations and arise specifically from the demands of iterative, stateful computation.

3.4.1 Premature Termination. Premature termination occurs when the agent declares task completion before reaching a goal state i.e. when $G(s') \neq \text{true}$, either by interpreting a partial success as full task completion or by defaulting to termination upon encountering an unexpected observation rather than reasoning about the error.

3.4.2 Action Loops. Action loops occur when the agent repeatedly executes the same or functionally equivalent actions, failing to progress toward the goal. This represents a failure of meta-cognitive monitoring, typically triggered by error observations that cause the model to retry the exact same action indefinitely.

Table 1: Taxonomy of agentic failure modes in small language models. Frequency (%) is the percentage of τ -bench Retail tasks where each failure type was observed (failures may co-occur). M1 = Mistral-7B-Instruct, M2 = Qwen3-14B, M3 = GPT-OSS-20B. Failure modes within the same dimension share a frequency measurement, except Agentic Reasoning, which is measured separately for (Premature Termination, Partial Completion) and (Action Loops, Recovery Failure). NLU and Task Decomposition/Sequencing consistently emerge as the two dominant failure dimensions across all model scales.

Dimension	Failure Mode	Description	M1	M2	M3
NLU	Intent Misclassification	Maps user query to wrong command f	90.00%	68.75%	61.11%
	Parameter Extraction Error	Produces invalid or incomplete θ			
Tool Management	Tool Overload	Fails to select from large action space $ F $	45.00%	30.00%	20.00%
	Schema Hallucination	Invokes non-existent tools or fabricates arguments			
Task Decomposition and Sequencing	Decomposition Failure	Cannot break multi-step goal into command sequence	60.00%	65.00%	45.00%
	Dependency Blindness	Ignores ordering constraints between sub-tasks			
Agentic Reasoning	Premature Termination	Declares task complete before $G(s') = \text{true}$	15.72%	36.00%	12.00%
	Partial Completion	Completes subset of sub-goals, silently drops others			
	Action Loops	Repeats identical actions without progress			
Context Management	Recovery Failure	Cannot adapt strategy after error observation	19.15%	12.00%	8.00%
	Trajectory Overflow	Context window exceeded, early information lost			
	State Tracking Failure	Loses track of executed operations and current w			

3.4.3 Recovery Failure. When a tool call returns an error observation o_{err} , the agent should adapt its strategy to recover from the failure. We observe three sub-patterns: *error blindness*, where the agent ignores the error entirely and proceeds as if the action succeeded, causing cascading failures in subsequent dependent actions; *strategy rigidity*, where the agent acknowledges the error but cannot generate an alternative approach, repeatedly attempting the same failed action or simply terminating due to an inability to reason about why the action failed and what alternative paths exist; and *catastrophic recovery*, where the agent attempts recovery but selects an action that worsens the state, particularly dangerous in workflows with irreversible operations.

3.4.4 Partial Completion. Partial completion occurs when a user’s request contains multiple sub-goals and the agent completes some but silently drops others, losing track of remaining sub-goals rather than incorrectly believing the task is complete. This failure correlates with task complexity and trajectory length.

3.5 Context Management Failures

Context management failures stem from the finite context window of transformer-based models and the growing trajectory length in multi-turn agentic interactions.

3.5.1 Trajectory Overflow. As the agent executes actions and accumulates observations, the conversation history h grows monotonically. Trajectory overflow occurs when $|h|$ approaches or exceeds the model’s context window limit L , leading to information loss. When early turns are truncated from the prompt, the agent may lose access to critical information: the original user request, previously extracted parameters, or the outcomes of earlier actions. This is particularly damaging for tasks that require coordinating information across many turns, as the agent effectively develops “amnesia” about its earlier actions. This effect is amplified for small

models, which have shorter context windows and lose capacity faster due to verbose tool responses/error messages.

3.5.2 State Tracking Failure. Even when the full trajectory fits within the context window, small models may fail to accurately track the current workflow state w as it evolves through tool executions. State tracking failure occurs when the agent’s internal representation of w diverges from the actual state. This manifests as redundant re-execution of completed operations, skipped steps, or decisions based on stale state. The root cause is that maintaining an accurate mental model of system state requires integrating information across multiple tool call responses, a reasoning task that scales in difficulty with trajectory length and that small models handle less reliably than their larger counterparts.

3.6 Interaction Between Failure Modes

In practice, failure modes rarely occur in isolation. We observe characteristic *failure cascades* in which one failure triggers others:

- (1) **NLU $\rightarrow$ Agentic:** An intent misclassification (Section 3.1.1) leads to an error observation. The model’s inability to recover (Section 3.4.3) leads to either action loops (Section 3.4.2) or premature termination (Section 3.4.1).
- (2) **Task Decomposition/Sequencing $\rightarrow$ Context:** Poor task decomposition (Section 3.3.1) leads to unnecessarily long trajectories, which accelerate trajectory overflow (Section 3.5.1).
- (3) **Tool $\rightarrow$ Task Decomposition/Sequencing:** Tool overload (Section 3.2.1) impairs the model’s ability to identify valid decomposition paths, compounding decomposition failures (Section 3.3.1).
- (4) **Context $\rightarrow$ NLU:** As trajectory overflow causes loss of earlier context (Section 3.5.1), parameter extraction degrades because prior user-provided values are no longer accessible (Section 3.1.2).

These cascades explain why small model performance degrades super-linearly with task complexity: each additional step not only introduces its own failure probability but also increases the likelihood of triggering compound failures across dimensions.

4 fastWorkflow Architecture

We now present fastWorkflow, an architectural framework designed to mitigate failure categories through targeted interventions. The design philosophy centers on a key insight: rather than relying solely on the agent’s reasoning capabilities, we can offload failure-prone operations to specialized subsystems preferably deterministic that provide guardrails while preserving the flexibility of agentic execution.

fastWorkflow operates in two phases. In the **training phase**, the framework prepares the NLU pipeline for the target workflow without requiring any manually labeled data. The intent detection classifiers are fine-tuned on synthetically generated utterance corpora, one per command context $c \in C$, and the escalation and ambiguity thresholds $\tau_{escalation}$ and $\tau_{ambiguity}$ are automatically calibrated from validation outputs. In parallel, command-specific few-shot examples for the parameter extraction module are generated. In the **run phase**, the trained classifiers and few-shot examples are loaded at workflow startup, and the system is ready to serve both assistant and agent mode requests with no further configuration.

4.1 Cascaded NLU Pipeline

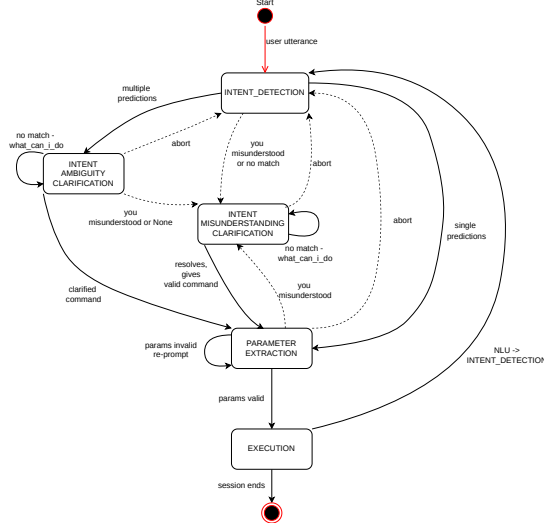


Figure 2: The NLU Pipeline State Machine

The NLU pipeline is implemented as a state machine (Figure 2) that orchestrates two critical operations: intent detection (mapping user utterances to workflow commands) and parameter extraction (extracting structured arguments for the detected command). This state machine design addresses both intent misclassification failures (Section 3.1.1) and parameter extraction failures (Section 3.1.2) through explicit error states and recovery mechanisms.

The pipeline operates through four distinct states, with transitions triggered by the outcome of processing at each stage. Unlike a simple cascade, this state machine can transition backwards for error recovery, and it explicitly handles ambiguous situations through dedicated clarification states rather than forcing premature commitment to a single interpretation. This explicit state management prevents the cascading failures observed when small models handle NLU implicitly through unconstrained generation.

4.1.1 Intent Detection. Given a user utterance u_t and current command context c_t , the intent detection proceeds through four stages, each activated only if the previous stage fails to produce a confident match.

Stages 1–3: Deterministic and Cached Matching. The first three stages provide fast, high-precision resolution without invoking learned models. Stage 1 performs an $O(1)$ exact-match lookup against command names in F_c . If no match is found, Stage 2 computes Levenshtein similarity against all commands in F_c , returning the best match when the ratio exceeds τ_{fuzzy} . Stage 3 consults a session-scoped embedding similarity cache that stores dense vectors and command mappings from prior user clarifications; when cosine similarity to a cached embedding exceeds τ_{cache} , the associated command is returned. This cache converts expensive clarification interactions into free training signal, enabling one-shot adaptation to user-specific terminology without offline retraining.

Stage 4: Two-Stage Classifier Cascade. When all prior stages fail, the system invokes a trained classifier cascade optimized for cost-accuracy tradeoff (Figure 3). The first stage employs a BERT-Tiny encoder with a linear classification head trained on synthetically-generated utterances per command context. When BERT-Tiny’s softmax confidence falls below an automatically-calibrated threshold $\tau_{escalation}$, the system escalates to a DistilBERT encoder trained on the same corpus. Both models return top- k ($k \leq 3$) predictions when confidence remains below $\tau_{ambiguity}$, triggering transition to Intent Ambiguity state. When no stage produces a confident match, the pipeline transitions to Intent Misunderstanding clarification state. In both states, a dedicated intent clarification agent attempts to resolve the ambiguity from trajectory context before surfacing it to the user (see Appendix A.5). The ambiguity threshold is calibrated per model based on the mean confidence of misclassified validation examples, providing a natural boundary between confident-and-correct versus confident-but-wrong predictions.

Context-Based Command Filtering. Each context $c \in C$ maintains its own command vocabulary $F_c \subset F_{total}$ and a separately-trained classifier M_c . All four detection stages operate exclusively over F_c , achieving $O(|F_c|)$ complexity and high accuracy on small, semantically-coherent command subsets. When no match is found, the system traverses the class inheritance hierarchy, re-invoking detection in parent contexts until a match is found or the root is reached (Section 4.2).

4.1.2 Parameter Extraction and Validation. Given a confirmed intent f and user utterance u_t , the system attempts extraction through two complementary methods (Figure 4).

Method 1: Structured Tag Extraction. In agent mode, the extractor applies regex pattern matching to extract XML-tagged values,

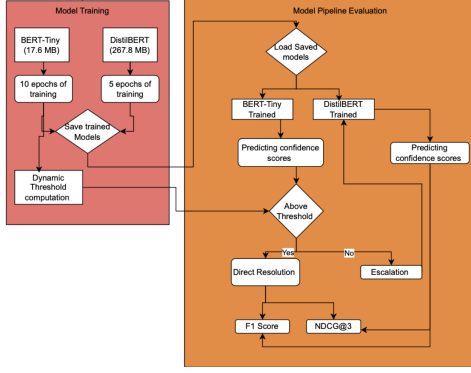


Figure 3: Two-Stage Classifier Cascade for Intent Detection.

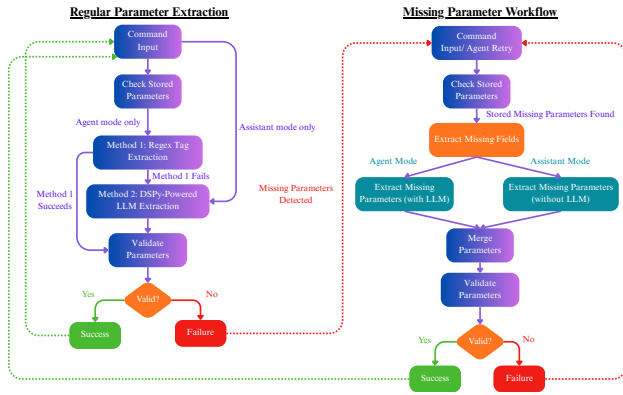


Figure 4: Parameter Extraction Pipeline with regular extraction (left) and missing parameter recovery (right).

matching field names explicitly for multi-parameter commands. This fast path commits only when *all* required parameters are extracted with high confidence.

Method 2: LLM-Based Extraction with Dynamic Signatures. When regex extraction fails or is inapplicable (assistant mode), the system invokes an LLM via a generalized DSPy [14] framework that dynamically generates a Signature from the command’s Pydantic schema at runtime. Synthetic few-shot examples, automatically generated and validated per command during the training phase, are injected via DSPy’s LabeledFewShot optimizer to improve domain-specific accuracy without manual annotation.

Parameter Value Preservation. When extraction produces an incomplete parameter set, missing fields are stored in persistent workflow context. On subsequent turns, extraction targets only those fields, and successfully extracted values are merged via selective overwrite. This transforms multi-turn parameter collection from an error-prone reasoning task into deterministic state accumulation.

Multi-Layer Validation and Error Recovery. Extracted parameters pass through three validation layers: (1) *type coercion*, which automatically converts values to declared types (string-to-integer, string-to-enum, string-to-list, etc.); (2) *constraint validation*, which

checks values against regex patterns and value ranges declared in Pydantic metadata; and (3) *semantic validation via database lookup*, which validates entity references using two-stage fuzzy matching and returns up to three suggestions on near-misses. When validation fails, the system generates structured error messages comprising a missing information block (sentinel-marked fields, with `available_from` hints directing the agent to prerequisite commands), an invalid information block (rejected values with reasons and format examples), and correction instructions. This structured format enables both human users and agentic systems to parse errors and formulate targeted corrections rather than blind retries.

4.2 Context Hierarchies for Tool Management

Tool management failures (Section 3.2) stem from exposing small models to large action spaces. fastWorkflow addresses this through hierarchical context organization that reduces the effective action space $|F_c|$ from potentially hundreds of commands to tens of contextually relevant ones, while maintaining access to shared functionality through two complementary mechanisms.

Command Inheritance. Commands are organized in directories mirroring the application’s class hierarchy. A declarative JSON configuration specifies inheritance relationships, and child contexts can override inherited commands for specialization. At workflow load time, the system pre-computes the effective command set F_c for each context via depth-first resolution of the inheritance graph, caching the result for $O(1)$ runtime lookup.

Context Navigation. Orthogonal to inheritance, a navigation model defines parent-child traversal relationships, enabling users to drill down into specific objects or navigate back to broader contexts. When a command cannot be resolved in the current context, the system traverses upward through the navigation hierarchy, checking ancestor command sets until a match is found or the root is reached. This ensures broadly-applicable commands remain accessible regardless of navigation depth.

4.3 Task Planning and Adaptive Replanning

fastWorkflow has an explicit planning phase followed by adaptive replanning when execution deviates from the plan.

Planning. A dedicated planner module which transforms user queries into structured step-by-step execution plans before the agent begins acting. The planner navigates an implicit dependency graph encoded in command metadata: `available_from` hints in parameter signatures specify which commands must execute first to provide required values, transforming dependency resolution from implicit reasoning into explicit graph traversal. The generated plan is injected alongside the original query into the workflow agent’s loop, offloading decomposition to a dedicated phase and making the intended execution sequence visible in the trajectory.

Adaptive Replanning. During execution, the system monitors three conditions that trigger trajectory-aware replanning: (1) *action loop detection*, where consecutive identical actions indicate perseveration on a failing strategy; (2) *parameter extraction errors*, where the agent fails to provide valid parameters; and (3) *user response*

integration, where new information from the user must be incorporated into the plan. When any trigger fires, the replanner receives the original query, the complete execution trajectory, and the trigger context, enabling it to perform counterfactual reasoning about what failed and generate a revised strategy that avoids repeating unsuccessful patterns.

4.4 Adaptive Context Management

fastWorkflow addresses context management failures (Section 3.5) through four complementary mechanisms: command catalog separation (injecting schemas into the system message rather than the trajectory), trajectory truncation (removing complete interaction cycles when approaching the context limit), state externalization (maintaining critical workflow state in persistent storage rather than reconstructing it from trajectory), and conversation history summarization (LLM-generated compact summaries injected at the start of subsequent turns). Details are provided in Appendix A.7.

5 Results

We evaluate fastWorkflow on the τ -bench benchmark, comparing small language models augmented with our framework against frontier model baselines. Our experiments demonstrate that GPT-OSS-20B with fastWorkflow achieves state-of-the-art performance on both benchmark domains, surpassing all frontier models on the public leaderboard while operating at a fraction of their inference cost. Additionally, we show that fastWorkflow’s architectural benefits extend to frontier models themselves.

5.1 Experimental Setup

5.1.1 Benchmark: τ -bench. We evaluate on τ -bench (Tool-Agent-User Interaction Benchmark), a challenging evaluation framework for agentic systems that simulates realistic multi-turn conversations requiring tool use. Tasks require coordinating multiple database operations while adhering to business rules and policy constraints. The benchmark comprises two domains: the **Retail domain**, a customer service simulation with 115 tasks spanning order management, returns processing, address modifications, and account inquiries; and the **Airline domain**, a flight booking and management simulation with 50 tasks covering reservation changes, cancellations, upgrades, and multi-leg itinerary modifications.

5.1.2 Evaluation Protocol. All experiments use GPT-5 as the user simulator, ensuring consistent and realistic user behavior across all agent configurations. We report Pass@1 (single-attempt success rate) as the primary metric, measuring the percentage of tasks completed correctly on the first attempt without human intervention. Task success requires that all user-requested tasks are executed correctly, the final database state matches the expected outcome, and no policy violations occur during execution.

5.1.3 Models Evaluated. We evaluate fastWorkflow on three open-weight models of varying scales: Mistral-7B-Instruct-v0.3 (7B)[11], Qwen3-14B (14B)[33], and GPT-OSS-20B (20B)[21]. For comparison, we include results from Anthropic’s blog[1, 2] for frontier models without our framework augmentation.

The frontier model baselines in τ -bench receive both structured tool schemas and domain-specific policy. The baseline is a standard

tool-calling agent loop: the model iteratively selects tools, receives observations, and continues until the task is resolved.

5.2 Main Results

5.2.1 τ -bench Performance. Figure 5 presents our main results on τ -bench with fastWorkflow-augmented small models compared to frontier model baselines from the public leaderboard. Multiple trial results are reported in Appendix B.4.

5.2.2 Key Findings.

Finding 1: fastWorkflow enables small models to match or exceed frontier performance. GPT-OSS-20B augmented with fastWorkflow achieves 83.47% on Retail and 78.00% on Airline, surpassing all frontier models on both domains including Claude Opus 4.1 (82.4% Retail, 56% Airline), while operating at a fraction of the inference cost.

Finding 2: Strongest gains on the Airline domain. The Airline domain, which requires strict dependency ordering and multi-step reasoning, shows the clearest advantage of structured orchestration. Mistral-7B + fastWorkflow matches Claude Sonnet 4 exactly at 60% and outperforms Claude Opus 4.1 (56.0%), despite being orders of magnitude smaller. GPT-OSS-20B + fastWorkflow further extends this to 78%, an absolute improvement of 18–22 percentage points over frontier models.

Finding 3: Base model capability interacts non-monotonically with the framework. Among fastWorkflow-augmented models, Qwen3-14B (43.47% Retail, 54% Airline) underperforms Mistral-7B (70.43% Retail, 60% Airline) despite having twice the parameters. This suggests that instruction-following quality and compatibility with the framework’s structured scaffolding matters more than raw parameter count within the small model regime.

Finding 4: Cost-performance Pareto improvement. fastWorkflow shifts the cost-performance frontier: GPT-OSS-20B with fastWorkflow achieves frontier-level performance (83.47% Retail, 78% Airline) at $\sim 22\times$ lower per-task cost compared to Claude Opus 4.1 on the τ -bench leaderboard configuration (\$0.017/task vs. \$0.375/task; see Appendix B.3). This makes high-quality agentic task completion viable for cost-sensitive and privacy-sensitive deployments.

Finding 5: Frontier models also benefit substantially from fastWorkflow. The improvements demonstrate that fastWorkflow’s architectural benefits are not limited to SLMs. Frontier models already handle intent detection and instruction following well, the gains come from structured feedback on parameter extraction errors, which means fewer wasted turns and reduced trajectory length overall. This limits context degradation. The second benefit comes from the dedicated planner module and the `available_from` dependency hints, which provide explicit dependency-aware decomposition both upfront and during execution. This benefits even capable models on complex multi-step tasks.

5.3 Ablation Studies

To isolate the contribution of each architectural component, we conduct two complementary analyses: (1) component ablation on both the Retail domain (Mistral-7B-Instruct) and Airline domain

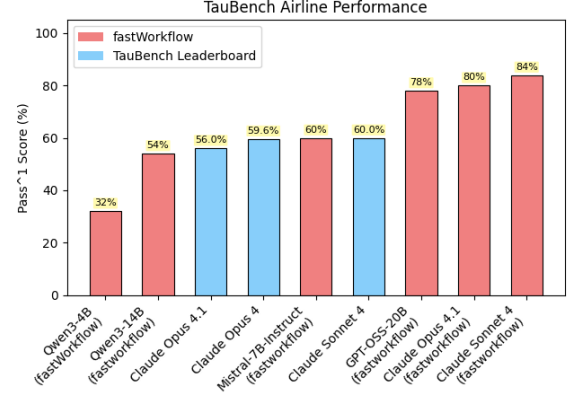
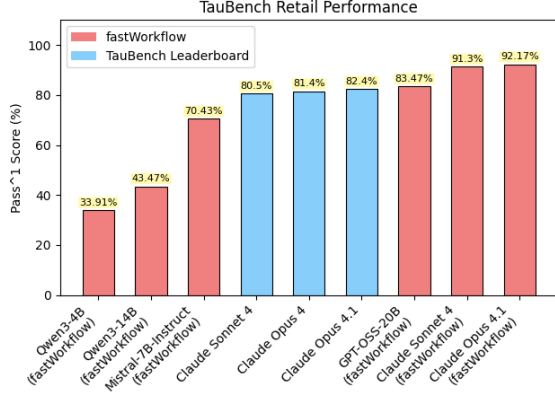


Figure 5: Comparison of Pass@1 scores on the τ -bench Retail (left) and Airline (right) domains. Models augmented with fastWorkflow (red bars) demonstrate competitive or superior performance compared to frontier models from the public leaderboard (blue bars).

(GPT-OSS-20B), where we remove key modules from the full system, and (2) incremental contribution analysis on the Retail domain using Mistral-7B-Instruct, where we measure the marginal gain from adding each component to a growing pipeline.

5.3.1 Component Ablation. Figure 6 shows the impact of removing the two core modules from the full fastWorkflow configuration (70.43% Pass@1).

Removing the Cascaded NLU Pipeline causes a near-total collapse on both domains: to 12.00% Pass@1 on Retail (−58.43% points) and 10% on Airline (−68% points). This confirms that NLU is the dominant bottleneck for small models in agentic settings: without structured intent detection and parameter extraction, neither Mistral-7B nor GPT-OSS-20B can reliably invoke even simple single-step commands, let alone complete multi-step tasks. The severity of this drop validates our taxonomy’s identification of NLU failures (Section 3.1) as the earliest and most consequential failure point in the agent pipeline.

Removing the Task Planner reduces Retail performance by 16.52% points to 53.91%, while on Airline the drop is far more severe: 62% points to 16%. The domain asymmetry is notable: the Airline domain’s strict dependency chains and multi-step booking constraints make explicit planning nearly as critical as NLU itself. This demonstrates that while the NLU pipeline is necessary, it is not sufficient and the value of explicit task decomposition scales with task complexity and dependency depth (Section 3.3).

5.3.2 Incremental Contribution Analysis. Table 2 reports the marginal improvement from progressively adding components to the pipeline. Each row shows the gain attributable to the indicated component.

The three components contribute roughly equally (+7.03, +6.97, and +7.00 points respectively), suggesting that no single mechanism dominates. Rather each addresses a distinct failure mode that the others cannot compensate for. Adaptive context management (trajectory summarization and placing the command catalog in the system message) prevents the agent from losing access to critical information as trajectories grow, directly mitigating trajectory

Table 2: Incremental contributions on τ -bench Retail (Mistral-7B). Each row reports the absolute Pass@1 improvement from adding the indicated component. Components are listed in the order they were integrated.

Component Added	Δ Pass@1
Adaptive Replanning	+6.97
Adaptive Context Management	+7.03
Insights Distillation	+7.00

overflow (Section 3.5.1). Replanning converts repetitive behavior into adaptive recovery, addressing action loops (Section 3.4.2) and recovery failures (Section 3.4.3). Insights distillation (Appendix A.8) closes residual reasoning gaps by encoding frontier-model behavioral patterns into the small model’s prompts, providing gains that are complementary to the structural mitigations.

The sustained marginal improvement at each stage where each component yields approximately +7 points even after the prior components are already in place indicates that substantial failure modes persist beyond what NLU and planning alone resolve. Since each gain was measured sequentially, the fact that later additions do not show diminishing returns suggests the targeted failure dimensions are largely independent rather than overlapping.

5.4 Discussion

Why NLU dominance matters. The ablation study reveals a striking asymmetry: removing the cascaded NLU pipeline causes a near-total collapse on both domains — 58.43 points on Retail and 68 points on Airline. This confirms that for small models, the bottleneck is not “what to do” but “how to do it”, translating correct intent into valid tool invocations. Frontier models absorb this burden implicitly through sheer capacity, but small scale models cannot reliably perform intent discrimination and schema-conformant parameter extraction within unconstrained generation. By offloading these operations to deterministic classifiers and structured extractors,

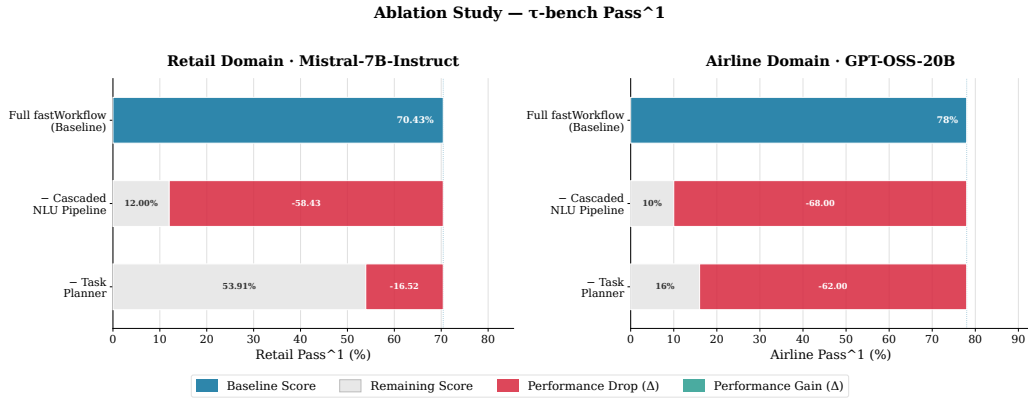


Figure 6: Component ablation on τ -bench Retail (Mistral-7B) and Airline (GPT-OSS-20B). Each row removes one component from the full system.

fastWorkflow eliminates the single largest source of failure cascades: an NLU error at step one propagates into recovery failures, action loops, and premature termination downstream (Section 3.6).

Residual failure modes. Despite fastWorkflow’s gains, a meaningful performance gap remains, particularly visible in Mistral-7B’s 70.43% Retail score versus the 83.47% achieved by GPT-OSS-20B with the same framework. Analysis of remaining failures reveals these categories: First, reasoning quality remains a binding constraint: small models still make errors in multi-step logical inference. These failures occur after correct intent detection and parameter extraction, within the planning, execution and recovery phases that remain under LLM control. Second, attentional misdirection persists: small models often fail to focus on task-relevant information in long trajectories. Third, working memory limitations surface in complex multi-goal requests where the model must track several concurrent sub-task states.

Sub-7B model limitations. We evaluated fastWorkflow on Qwen3-4B, which achieves 33.91% Pass¹ on Retail and 32% on Airline. While this represents meaningful task completion at a scale where performance is generally expected to be very limited, performance remains well below both frontier models and the 7B–20B range. The core limitations at this scale are twofold. First, feedback incorporation breaks down: when the NLU pipeline or execution layer returns structured error messages with correction instructions, such smaller models frequently fail to parse and act on this feedback, instead repeating the same erroneous action or generating malformed responses. Additionally, function calling ability degrades significantly: the model struggles to produce well-formed arguments to call tools. At the 4B scale and below, failures are more likely to be unrecoverable: the feedback loops that allow 7B+ models to self-correct within the fastWorkflow scaffolding simply do not engage reliably. For practical edge deployment, the most viable path for sub-7B models is restricting to assistant mode only: single-turn command execution through the NLU pipeline without the agent reasoning loop, relying entirely on the deterministic cascaded pipeline for intent detection and parameter extraction.

6 Conclusion and Future Work

Our results confirm that dominant failure modes of small models in conversational workflow settings can be systematically resolved by using structured subsystems. The empirically-grounded failure taxonomy identified five interacting failure dimensions whose characteristic cascades cause non-linear performance degradation with task complexity, guiding targeted interventions over monolithic scaling. The fastWorkflow framework separates these concerns: subsystems handle intent detection, parameter extraction, and state tracking while the LLM retains planning and recovery flexibility. This hybrid architecture leverages the strengths of both structured and neural components. Notably, the Airline domain in τ -bench demanding complex multi-step reasoning and strict dependency ordering showed the largest performance gain favoring fastWorkflow over frontier models, suggesting structured orchestration provides its greatest advantage precisely where failure cascades compound most severely. These findings demonstrate that harness-augmented small models offer a viable and often superior alternative to frontier models for conversational workflows, with lower latency, deployment flexibility, and privacy benefits, highlighting architectural design as a complementary path to scaling for reliable agents in domains involving multi-turn, tool-augmented human-LLM collaboration.

Several directions remain open. Our evaluation is scoped to conversational workflows; generalization to fully autonomous agent settings, code generation, and multimodal scenarios remains unvalidated. *Long context reasoning and long horizon planning* require retrieval mechanisms that are effective within small models’ tight token budgets, selectively injecting relevant context rather than naively appending to the context. *Domain generalization* beyond the customer service domains of τ -bench to structurally different tasks such as code generation, research assistance, and creative workflows remains to be validated. *Automated workflow adaptation*: generating command schemas, context hierarchies, etc. from natural language descriptions would reduce manual setup overhead and broaden adoption. Finally, *multi-agent coordination*, with specialized agents for planning, execution, and orchestration, is a natural architectural extension.

Acknowledgments

Author Contributions. Sanchit Satija led the research and was responsible for the overall system design, agent optimization, parameter extraction pipeline, experimental evaluation, and paper writing. Aditya Bhatt designed and implemented the intent detection component. Priyanshu Jani contributed to experimental evaluation and results. Dhar Rawal is the principal investigator who provided research guidance and mentorship. The authors would also like to thank Rakshit Pandhi for assistance with figure creation and the related work section.

References

- [1] Anthropic. 2025. Claude Opus 4.1. <https://www.anthropic.com/news/claude-opus-4-1>
- [2] Anthropic. 2025. Claude Sonnet 4. <https://www.anthropic.com/news/claude-4>
- [3] Peter Belcak, Greg Heinrich, Shizhe Diao, Yonggan Fu, Xin Dong, Saurav Muradharan, Yingyan Celine Lin, and Pavlo Molchanov. 2025. Small Language Models are the Future of Agentic AI. *arXiv:2506.02153 [cs.AI]* doi:10.48550/arXiv.2506.02153
- [4] Iñigo Casanueva, Tadas Temčinas, Daniela Gerz, Matthew Henderson, and Ivan Vulić. 2020. Efficient Intent Detection with Dual Sentence Encoders. In *Proceedings of the 2nd Workshop on Natural Language Processing for Conversational AI*, Tsung-Hsien Wen, Asli Celikyilmaz, Zhou Yu, Alexandros Papangelis, Mihail Eric, Anuj Kumar, Iñigo Casanueva, and Rushin Shah (Eds.). Association for Computational Linguistics, Online, 38–45. doi:10.18653/v1/2020.nlp4convai-1.5
- [5] Lutfi Eren Erdogan, Nicholas Lee, Siddharth Jha, Sehoon Kim, Ryan Tabrizi, Suhong Moon, Coleman Richard Charles Hooper, Gopala Anumanchipalli, Kurt Keutzer, and Amir Gholami. 2024. TinyAgent: Function Calling at the Edge. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, Delia Irazu Hernandez Farias, Tom Hope, and Manling Li (Eds.). Association for Computational Linguistics, Miami, Florida, USA, 80–88. doi:10.18653/v1/2024.emnlp-demo.9
- [6] Yao Fu, Dong-Ki Kim, Jaekyeom Kim, Sungrull Sohn, Lajanugen Logeswaran, Kyunghoon Bae, and Honglak Lee. 2024. AutoGuide: Automated Generation and Selection of Context-Aware Guidelines for Large Language Model Agents. In *Advances in Neural Information Processing Systems*, A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang (Eds.), Vol. 37. Curran Associates, Inc., 119919–119948. doi:10.52202/079017-3811
- [7] Suriya Gunasekar, Yi Zhang, Jyoti Aneja, Caio Cesar, Teodoro Mendes, Alie Del Giorno, Sivakanth Gopi, Mojan Javaheripi, Piero Kauffmann, Gustavo de Rosa, Olli Saarikivi, Adil Salim, Shital Shah, Harkirat Singh Behl, Xin Wang, Sébastien Bubeck, Ronen Eldan, Adam Tauman Kalai, Yin Tat Lee, and Yuanzhi Li. 2023. Textbooks Are All You Need. <https://www.microsoft.com/en-us/research/publication/textbooks-are-all-you-need/>
- [8] Shibo Hao, Yi Gu, Haodi Ma, Joshua Hong, Zhen Wang, Daisy Wang, and Zhiting Hu. 2023. Reasoning with Language Model is Planning with World Model. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, Houda Bouamor, Juan Pino, and Kalika Bali (Eds.). Association for Computational Linguistics, Singapore, 8154–8173. doi:10.18653/v1/2023.emnlp-main.507
- [9] Jianliang He, Siyu Chen, Fengzhuo Zhang, and Zhuoran Yang. 2024. From Words to Actions: Unveiling the Theoretical Underpinnings of LLM-Driven Autonomous Systems. In *Proceedings of the 41st International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 235)*, Ruslan Salakhutdinov, Zico Kolter, Katherine Heller, Adrian Weller, Nuria Oliver, Jonathan Scarlett, and Felix Berkenkamp (Eds.). PMLR, 17807–17841. <https://proceedings.mlr.press/v235/he24a.html>
- [10] Jie Huang, Xinyun Chen, Swaroop Mishra, Huaixiu Steven Zheng, Adams Wei Yu, Xinying Song, and Denny Zhou. 2024. Large Language Models Cannot Self-Correct Reasoning Yet. In *The Twelfth International Conference on Learning Representations*. <https://openreview.net/forum?id=lkmD3fKBPQ>
- [11] Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, Léo Renard Lavaud, Marie-Anne Lachaux, Pierre Stock, Teven Le Scao, Thibaut Lavril, Thomas Wang, Timothée Lacroix, and William El Sayed. 2023. Mistral 7B. *arXiv:2310.06825 [cs.CL]* <https://arxiv.org/abs/2310.06825>
- [12] Huiqiang Jiang, Qianhui Wu, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. 2023. LLMingua: Compressing Prompts for Accelerated Inference of Large Language Models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP 2023)*. <https://www.microsoft.com/en-us/research/publication/llmingua-compressing-prompts-for-accelerated-inference-of-large-language-models/>
- [13] Minki Kang, Jongwon Jeong, Seanie Lee, Jaewoong Cho, and Sung Ju Hwang. 2025. Distilling LLM Agent into Small Models with Retrieval and Code Tools. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*. <https://openreview.net/forum?id=VkcTQszOn>
- [14] Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan A, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. 2024. DSPy: Compiling Declarative Language Model Calls into State-of-the-Art Pipelines. In *The Twelfth International Conference on Learning Representations*. <https://openreview.net/forum?id=sY5N0zY5Od>
- [15] Stefan Larson, Anish Mahendran, Joseph J. Peper, Christopher Clarke, Andrew Lee, Parker Hill, Jonathan K. Kummerfeld, Kevin Leach, Michael A. Laurenzano, Lingjia Tang, and Jason Mars. 2019. An Evaluation Dataset for Intent Classification and Out-of-Scope Prediction. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*. <https://www.aclweb.org/anthology/D19-1131>
- [16] Zekun Li, Zhiyu Zoey Chen, Mike Ross, Patrick Huber, Seungwhan Moon, Zhaojiang Lin, Lina Dong, Adithya Sagar, Xifeng Yan, and Paul A. Crook. 2024. Large Language Models as Zero-shot Dialogue State Tracker through Function Calling. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Lun-Wei Ku, Andre Martins, and Vivek Srikumar (Eds.). Association for Computational Linguistics, Bangkok, Thailand, 8688–8704. doi:10.18653/v1/2024.acl-long.471
- [17] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2024. Lost in the Middle: How Language Models Use Long Contexts. *Transactions of the Association for Computational Linguistics* 12 (2024), 157–173. doi:10.1162/tac1_a_00638
- [18] Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Zhang, Shudan Zhang, Xiang Deng, Aohan Zeng, Zhengxiao Du, Chenhui Zhang, Sheng Shen, Tianjun Zhang, Yu Su, Huan Sun, Minlie Huang, Yuxiao Dong, and Jie Tang. 2024. AgentBench: Evaluating LLMs as Agents. In *The Twelfth International Conference on Learning Representations*. <https://openreview.net/forum?id=zAdUB0aCTQ>
- [19] Zechun Li, Changsheng Zhao, Forrest Iandola, Chen Lai, Yuandong Tian, Igor Fedorov, Yinyang Xiong, Ernie Chang, Yangyang Shi, Raghuraman Krishnamoorthi, Liangzhen Lai, and Vikas Chandra. 2024. MobileLLM: Optimizing Sub-billion Parameter Language Models for On-Device Use Cases. In *Proceedings of the 41st International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 235)*, Ruslan Salakhutdinov, Zico Kolter, Katherine Heller, Adrian Weller, Nuria Oliver, Jonathan Scarlett, and Felix Berkenkamp (Eds.). PMLR, 32431–32454. <https://proceedings.mlr.press/v235/liu24ce.html>
- [20] Arindam Mitra, Luciano Del Corro, Shweti Mahajan, Andres Coda, Clarisse Simoes Ribeiro, Sahaj Agrawal, Xuxi Chen, Anastasia Razdaibiedina, Erik Jones, Kriti Aggarwal, Hamid Palangi, Guoqing Zheng, Corby Rosset, Hamed Khanpour, and Ahmed Awadallah. 2023. Orca-2: Teaching Small Language Models How to Reason. *arXiv*. <https://www.microsoft.com/en-us/research/publication/orca-2-teaching-small-language-models-how-to-reason/>
- [21] OpenAI. 2025. gpt-oss-120b & gpt-oss-20b Model Card. *arXiv:2508.10925 [cs.CL]* <https://arxiv.org/abs/2508.10925>
- [22] Melissa Z. Pan, Mert Cemri, Lakshya A. Agrawal, Shuyi Yang, Bhavya Chopra, Rishabh Tiwari, Kurt Keutzer, Aditya Parameswaran, Kannan Ramchandran, Dan Klein, Joseph E. Gonzalez, Matei Zaharia, and Ion Stoica. 2025. Why Do Multiagent Systems Fail?. In *ICLR 2025 Workshop on Building Trust in Language Models and Applications*. <https://openreview.net/forum?id=wM521FqPv1>
- [23] Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. 2024. Gorilla: Large Language Model Connected with Massive APIs. In *Advances in Neural Information Processing Systems*, A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang (Eds.), Vol. 37. Curran Associates, Inc., 126544–126565. doi:10.52202/079017-4020
- [24] Thinh Pham, Chi Tran, and Dat Quoc Nguyen. 2023. MISCA: A Joint Model for Multiple Intent Detection and Slot Filling with Intent-Slot Co-Attention. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, Houda Bouamor, Juan Pino, and Kalika Bali (Eds.). Association for Computational Linguistics, Singapore, 12641–12650. doi:10.18653/v1/2023.findings-emnlp.841
- [25] Aske Plaat, Max Van Duijn, Niki Van Stein, Mike Preuss, Peter Van der Putten, and Kees Joost Batenburg. 2025. Agentic Large Language Models, a Survey. *Journal of Artificial Intelligence Research* 84 (Dec. 2025). doi:10.1613/jair.1.18675
- [26] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, dahai li, Zhiyuan Liu, and Maosong Sun. 2024. ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs. In *The Twelfth International Conference on Learning Representations*. <https://openreview.net/forum?id=dHng2O0Jjr>
- [27] Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023.

- Toolformer: Language Models Can Teach Themselves to Use Tools. In *Advances in Neural Information Processing Systems*, A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine (Eds.), Vol. 36. Curran Associates, Inc., 68539–68551. https://proceedings.neurips.cc/paper_files/paper/2023/file/d842425e4bf79ba039352da0f658a906-Paper-Conference.pdf
- [28] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine (Eds.), Vol. 36. Curran Associates, Inc., 8634–8652. https://proceedings.neurips.cc/paper_files/paper/2023/file/1b44b878bb782e6954cd888628510e90-Paper-Conference.pdf
- [29] Gaurav Srivastava, Aafiya Hussain, Chi Wang, Yingyan Celine Lin, and Xuan Wang. 2026. EffGen: Enabling Small Language Models as Capable Autonomous Agents. arXiv:2602.00887 [cs.CL] <https://arxiv.org/abs/2602.00887>
- [30] Hongjin Su, Shizhe Diao, Ximing Lu, Mingjie Liu, Jiacheng Xu, Xin Dong, Yonggan Fu, Peter Belcak, Hanrong Ye, Hongxu Yin, Yi Dong, Evelina Bakhturina, Tao Yu, Yejin Choi, Jan Kautz, and Pavlo Molchanov. 2025. ToolOrchestra: Elevating Intelligence via Efficient Model and Tool Orchestration. arXiv:2511.21689 [cs.CL] <https://arxiv.org/abs/2511.21689>
- [31] Shreyas Subramanian, Vikram Elango, and Mecit Gungor. 2025. Small Language Models (SLMs) Can Still Pack a Punch: A survey. arXiv:2501.05465 [cs.CL] <https://arxiv.org/abs/2501.05465>
- [32] Lei Wang, Wanyu Xu, Yihuai Lan, Zhiqiang Hu, Yunshi Lan, Roy Ka-Wei Lee, and Ee-Peng Lim. 2023. Plan-and-Solve Prompting: Improving Zero-Shot Chain-of-Thought Reasoning by Large Language Models. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Anna Rogers, Jordan Boyd-Graber, and Naoaki Okazaki (Eds.). Association for Computational Linguistics, Toronto, Canada, 2609–2634. doi:10.18653/v1/2023.acl-long.147
- [33] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jing Zhou, Jingren Zhou, Junyang Lin, Kai Dang, Keqin Bao, Kexin Yang, Le Yu, Lianghao Deng, Mei Li, Mingfeng Xue, Mingze Li, Pei Zhang, Peng Wang, Qin Zhu, Rui Men, Ruize Gao, Shixuan Liu, Shuang Luo, Tianhao Li, Tianyi Tang, Wenbiao Yin, Xingzhang Ren, Xinyu Wang, Xinyu Zhang, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yinger Zhang, Yu Wan, Yuqiong Liu, Zekun Wang, Zeyu Cui, Zhenru Zhang, Zhipeng Zhou, and Zihan Qiu. 2025. Qwen3 Technical Report. arXiv preprint arXiv:2505.09388 (2025).
- [34] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. 2024. τ -bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. arXiv:2406.12045 [cs.AI] <https://arxiv.org/abs/2406.12045>
- [35] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik R Narasimhan. 2023. Tree of Thoughts: Deliberate Problem Solving with Large Language Models. In *Thirty-seventh Conference on Neural Information Processing Systems*. <https://openreview.net/forum?id=5Xc1ecxOIh>
- [36] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations (ICLR)*. <https://arxiv.org/abs/2210.03629>
- [37] Da Yin, Faeze Brahman, Abhilasha Ravichander, Khyathi Chandu, Kai-Wei Chang, Yejin Choi, and Bill Yuchen Lin. 2024. Agent Lumos: Unified and Modular Training for Open-Source Language Agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Lun-Wei Ku, Andre Martins, and Vivek Srikumar (Eds.). Association for Computational Linguistics, Bangkok, Thailand, 12380–12403. doi:10.18653/v1/2024.acl-long.670
- [38] Muru Zhang, Ofir Press, William Merrill, Alisa Liu, and Noah A. Smith. 2023. How Language Model Hallucinations Can Snowball. arXiv:2305.13534 [cs.CL] <https://arxiv.org/abs/2305.13534>
- [39] Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. 2024. ExpeL: LLM Agents Are Experiential Learners. *Proceedings of the AAAI Conference on Artificial Intelligence* 38, 17 (Mar. 2024), 19632–19642. doi:10.1609/aaai.v38i17.29936

A fastWorkflow Details and Extended Methodologies

A.1 Extended Problem Formalization

This section formalizes the agentic task completion problem, defines what constitutes a “small” language model in this context, and establishes the mathematical framework for analyzing failure modes.

A.1.1 State Space. The state $s \in S$ at any point in the interaction is a tuple:

$$s = (h, w, c) \quad (1)$$

where:

- $h = (u_1, a_1, o_1, \dots, u_t, a_t, o_t)$ is the conversation history, a sequence of user utterances u_i , agent actions a_i , and observations o_i
- $w \in W$ is the workflow state (e.g., database records, user session data)
- $c \in C$ is the current command context determining which workflow commands are available for execution

The state is only partially observable to the agent; specifically, w is observed only through tool call responses.

A.1.2 Action Space. The agent interacts with the environment through a set of **meta-tools** M that provide different capabilities. Each meta-tool $m \in M$ has its own argument schema.

The action space A is therefore:

$$A = \bigcup_{m \in M} A_m \quad (2)$$

where $A_m = \{(m, \theta) : \theta \in \Theta_m\}$ is the set of valid invocations of meta-tool m with arguments conforming to its schema Θ_m .

Each workflow command $f \in F$ (accessible via workflow query tool) has an associated **signature** defining:

- Required parameters: $\theta_{req} \subseteq \Theta_f$ (must be provided)
- Optional parameters: $\theta_{opt} \subseteq \Theta_f$ (may be omitted)
- Type constraints: $\tau : \Theta_f \rightarrow \text{Types}$ (e.g., int, string, enum)
- Value constraints: $V_f \subseteq \Theta_f \rightarrow \text{Predicates}$ (e.g., regex patterns, valid ranges)

Agent then routes the user query through an NLU pipeline to invoke the appropriate workflow command $f \in F$ with extracted parameters θ .

A.1.3 Transition Dynamics. The transition function $T : S \times A \rightarrow \Delta(S)$ captures how meta-tool executions change system state:

- (1) **Meta-tool execution:** For action $a = (m, \theta)$, the environment executes meta-tool $m(\theta)$ and returns observation o
- (2) **State update:** Workflow state w may be modified (only for workflow query tool with successful write operations)
- (3) **Context update:** Command context c may change (e.g., after context-switching commands)
- (4) **History extension:** $h \leftarrow h \oplus (a, o)$

Formally, for meta-tool action (m, θ) (where prime notation denotes the next/updated version of a variable):

$$T(s, (m, \theta)) = P(s' \mid s, m, \theta) \quad \text{where} \quad s' = (h \oplus (m, \theta, o), w', c') \quad (3)$$

The state transition depends on the meta-tool. For execute workflow query tool, execution may succeed or fail:

- **Success:** o contains the command result, w updated if applicable
- **NLU Failure:** o contains intent detection or parameter extraction error, w unchanged
- **Execution Failure:** o contains error message (constraint violation, database error), w unchanged

A.1.4 Observation Space and Function. The observation space Ω consists of all possible meta-tool execution results. Each meta-tool $m \in M$ produces observations from a distinct subspace.

The observation function $O : S \times A \rightarrow \Delta(\Omega)$ determines which observation the agent receives after taking action $a = (m, \theta)$ in state s :

$$O(s, (m, \theta)) \rightarrow o \in \Omega_m \quad (4)$$

For workflow query tool specifically, the observation o is sampled from:

- Success response if the NLU pipeline resolves intent, extracts valid parameters, and $f(\theta)$ executes successfully
- Intent detection error if the command cannot be mapped to a workflow command
- Parameter extraction error if required parameters are missing or invalid
- Execution error if the workflow command fails (constraint violations, database errors, etc.)

These observations $o \in \Omega$ are the observation instances o_i that get appended to the conversation history h .

Task completion requires:

- (1) All user-requested tasks completed successfully
- (2) Final state satisfies goal predicate: $G(s') = \text{true}$

A.2 Failure Mode Labeling Methodology

Failure mode labeling used a combination of three methods: automated heuristics, deterministic execution trace analysis, and LLM-as-a-judge; selected per failure type based on what each mode's structure permits. The key design choice is that each failure dimension was assessed independently through its own dedicated detection pipeline, rather than through a single unified labeling pass. This means a given trajectory can receive labels across multiple dimensions simultaneously without any prioritization or mutual exclusion. An NLU failure and a planning failure and an action loop can all be flagged on the same task, which is why the frequencies in Table 1 sum to well over 100%. There is no ranking or assignment conflict because each detector operates on the trajectory in isolation, examining only the signals relevant to its own failure type.

A.2.1 NLU Failures. NLU failure modes were identified through automated heuristics applied to execution trajectories. We introduced a BYPASS_NLU environment flag as an ablation condition; under normal (non-bypass) runs, two signals in the trajectory were used to flag NLU failures: first, a required parameter populated with a NOT_FOUND sentinel value, indicating the agent failed to extract a valid argument; second, the command name resolved from the agent's output not matching any available command in the current

workflow context or was incorrect based on user utterance, indicating that the agent could not correctly map the user’s utterance to a valid/correct command. Both signals are evaluated on what the agent produced in response to the user utterance.

A.2.2 Tool Management Failures. Tool management failures were detected by examining the agent’s raw generated output directly. A tool management failure is flagged when the agent’s raw output contains a syntactically plausible but incorrect XML structure: e.g., using `<zip_code>` instead of the schema-mandated `<postal_code>`. A failure is also flagged when the agent-issued command name does not appear in the registered command set or is a malformed command name. At each step, fastWorkflow logs both the command name the agent attempted to invoke and the command the NLU actually dispatched.

A.2.3 Task Decomposition and Sequencing Failures. Task Decomposition and Sequencing failures were labeled using an LLM-as-a-judge (GPT-5.2). After each task, the judge received: the original user request (task instruction), the full user-agent conversation embedded in the ReAct trajectory (thoughts, tool calls, and observations across all turns), the domain policy wiki, the available command list, and the ground-truth action sequence. The judge assessed two failure modes: Task Decomposition Failure (wrong or incomplete sub-goal sequencing, per policy) and Dependency Blindness (hallucinating IDs or skipping required lookups before mutations), returning a boolean verdict, sub-type, and reasoning for each. The domain policy served as the authoritative reference for what a correct plan entails, since ground-truth action sequences are often terse and omit policy-mandated prerequisite steps.

A.2.4 Action Loops. Action loops were detected via an automated heuristic built into the fastWorkflow execution layer. After every tool call, the framework appends a record of the command name, parameters, success flag, and response snippet to a per-session action history. If the same command with identical parameters and the same outcome appears 2 or more times consecutively, a loop is flagged.

A.2.5 Recovery Failures. Recovery failures were identified using an LLM-as-a-judge approach. Specifically, the judge examined agent trajectories for instances where the system had received explicit corrective feedback: either a structured parameter extraction error from the NLU pipeline (e.g., missing or invalid fields with correction instructions) or a command execution error (e.g., invoking a return command when the correct action was modify), and the agent nevertheless failed to incorporate that feedback in its subsequent action. Cases where the agent repeated the same erroneous invocation, ignored the error observation entirely, or selected an equally incorrect alternative command were labeled as recovery failures.

A.2.6 Premature Termination / Partial Completion. Premature termination and partial completion (goal drop) failures were labeled using an LLM-as-a-judge (GPT-5.2). After each task, the judge received the original user request, the agent’s full trajectory, and a summary of the write operations (state-mutating tool calls) the agent actually executed. The judge was asked to determine whether

the agent silently dropped any write sub-goal that the user explicitly requested i.e., never attempted a state-mutating action for it at all. Crucially, if the agent attempted a write operation but failed due to wrong parameters, or if the agent attempted to address a sub-goal whether by issuing the correct write command or even an incorrect one, that was not counted as a drop; only complete omission of a required sub-goal without any attempt was flagged. The judge returned a boolean verdict, a list of dropped sub-goals, and a reasoning statement citing specific trajectory steps.

A.2.7 Context Management Failures. Trajectory overflow was detected by tracking the token count of the agent’s active context window at each ReAct step. Turns where the token count approached or exceeded the model’s context limit were flagged as overflow events. State tracking failure was identified by exploiting the fact that the agent’s trajectory is flushed at the end of each turn (i.e., after each finish call). On the subsequent turn, the agent begins with no direct memory of prior executed actions or their outcomes. We labeled a turn as a state tracking failure when the agent’s behavior indicated it had lost awareness of what had already transpired. For example, attempting to re-execute a command that had already succeeded, or contradicting a database state it had itself modified in a prior turn.

A.3 Dual-Mode Architecture Overview

fastWorkflow implements a dual-mode architecture where an LLM-based **workflow agent** operates in conjunction with an **assistant**. The assistant implements a natural language understanding (NLU) pipeline that handles intent detection and parameter extraction (operations that are particularly error-prone for small models) while the agent retains responsibility for high-level planning and multi-step reasoning.

A.3.1 Assistant Mode. In assistant mode, the system operates as a single-turn command executor. User input flows through a fixed pipeline: The assistant handles all NLU operations without agent

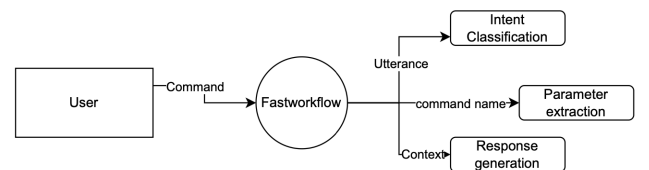


Figure 7: Context flow diagram for command execution in assistant mode

involvement, providing fast, predictable execution.

Assistant mode is well-suited for applications where user intent is largely predictable and tasks are self-contained within a single turn. Consider a customer service kiosk where a user says “cancel my order 4821”. In assistant mode, the utterance flows directly through the NLU pipeline: the intent detection cascade resolves the command to `cancel_order`, the parameter extraction module extracts `order_id = 4821`, validation confirms the value against the database, and the command executes — all without invoking an LLM

agent. This makes assistant mode significantly faster and cheaper than agent mode, as it avoids the overhead of a agent loop entirely. It is the preferred execution path whenever the user's request maps cleanly to a single workflow command, and fastWorkflow defaults to it before escalating to agent mode for requests that require multi-step reasoning or clarification.

A.3.2 Agent Mode. In agent mode, the system supports multi-turn, goal-oriented interaction. The agent operates in a custom ReAct loop, selecting from meta-tools. Critically, even in agent mode, the NLU pipeline handles intent detection and parameter extraction through the assistant. The agent decides *which* commands to attempt, but the assistant determines *how* those commands map to workflow operations.

A.4 Meta-Tools for fastWorkflow Workflow Agent

The fastWorkflow execution agent interacts with the system and the user through a tightly scoped set of meta-tools. These tools abstract away the complexity of the underlying workflow execution, NLU pipeline, and state management.

A.4.1 `execute_workflow_query(command: str)`. The primary tool for executing domain-specific workflow commands.

- **Purpose:** Executes a workflow command and returns either a response or an error message.
- **Input Format:** Plain text command name followed by XML-tagged parameters.
- **Key Features:**
 - Automatically traces command execution (logs raw commands and responses).
 - Maintains action history for loop detection.
 - Handles specialized error states (intent ambiguity, parameter extraction failures).
 - Integrates with specialized intent clarification agents when the wrong command is detected.
 - Triggers replanning with next steps generation when errors occur.

A.4.2 `what_can_i_do()`. Command discovery and exploration tool.

- **Purpose:** Returns a formatted list of all available commands in the current context, including their names and parameter specifications.
- **Output:** Structured metadata showing command signatures, parameter types, descriptions, and examples.
- **Use Case:** Helps the agent understand what operations are possible in the current workflow context.

A.4.3 `intent_misunderstood()`. Intent correction and self-recovery tool.

- **Purpose:** Provides the full command list when the agent detects that the wrong command was executed.
- **Trigger:** Called when the agent realizes its intent was misunderstood (e.g., wrong command name chosen).
- **Use Case:** Enables the agent to self-correct.

A.4.4 `ask_user(clarification_request: str)`. Human in the loop clarification tool.

- **Purpose:** Requests some information from the human user.
- **Input:** Plain text clarification question.
- **Key Features:**
 - Blocks agent execution until user responds.
 - Logs all agent-user dialogs to `action.jsonl`.
 - Stores raw user input in workflow context for downstream use.
 - Automatically generates next steps plan based on user's response.

A.5 Intent Clarification Agent

When the NLU pipeline enters either the Intent Ambiguity Clarification state or Intent Misunderstanding Clarification state, control is transferred to a dedicated intent clarification agent rather than surfacing the error directly to the main workflow agent. This separation prevents the main workflow agent's trajectory from being polluted with NLU recovery logic, keeping planning and intent resolution as distinct concerns.

The clarification agent is a lightweight fastWorkflow ReAct module operating over a restricted tool set. It receives four inputs: the original command that triggered the error, the intent detection error message, the main agent's original inputs, and the full agent trajectory up to the point of failure. The trajectory is critical, it allows the clarification agent to understand the broader task context and resolve ambiguity without asking the user for information that has already been established.

The agent's behavior differs by state. In the Intent Ambiguity Clarification state, it is additionally provided with the metadata of the top-*k* candidate commands returned by the classifier, including each command's purpose, parameters, and usage, and attempts to resolve the ambiguity by reasoning over this metadata in light of the agent trajectory before resorting to user interaction. In the Intent Misunderstanding clarification state, no candidate commands are available, and the agent reasons over the full available command set to identify the most plausible match. In both cases, the `ask_user` tool is available as a last resort when ambiguity cannot be resolved from context alone.

Once the correct command is identified, the agent returns a *clarified command* that preserves all parameter values from the original invocation, correcting only the command name. This clarified command is passed back to the main agent's execution pipeline, which resumes from the parameter extraction stage without replanning.

A.6 Elaborated Details on Context Hierarchies for Tool Management

A.6.1 Command Inheritance Model. The command inheritance model defines which commands are shared across contexts through a declarative inheritance hierarchy. Each workflow defines these relationships in a context inheritance model json file that mirrors the class hierarchy of the underlying application.

The inheritance model operates on three principles as below. Throughout this section, we use the `messaging_app_4` example from the fastWorkflow repository to illustrate these concepts concretely.

Directory-Based Context Binding. Commands are organized in directories that correspond to the application’s class structure. A command file placed in `_commands/User/` is bound to the User context; a file in `_commands/PremiumUser/` is bound to PremiumUser. This directory structure provides an intuitive mapping: the physical organization of command files mirrors the logical organization of the application’s object model.

Declarative Inheritance Specification. The context inheritance model json file declares inheritance relationships using a base array for each context. The special value “*” indicates a root context with no inheritance; otherwise, the array lists parent contexts from which commands are inherited. For example, `{"PremiumUser": {"base": ["User"]}}` declares that PremiumUser inherits all commands from User, a PremiumUser can invoke both `send_message` (inherited) and `send_priority_message` (own), while a regular User can only invoke `send_message`.

Polymorphic Command Access. When a command is invoked, the system resolves it against the current context’s effective command set, which includes both own commands and inherited commands. Commands defined in a child context override inherited commands of the same name, enabling specialization: a PremiumUser could define its own `send_message` command that differs from the inherited version. This polymorphic behavior mirrors object-oriented dispatch, providing natural semantics for users familiar with class hierarchies.

A.6.2 Context Navigation Model. The context navigation model defines parent-child relationships that enable bidirectional traversal through the application’s object hierarchy. This model is orthogonal to command inheritance: while inheritance determines which commands are *available*, navigation determines how users *move* between contexts.

Navigation relationships are defined through two mechanisms:

Declarative Parent Specification. The context hierarchy model json file declares parent-child relationships. For example, `{"User": {"parent": ["ChatRoom"]}}` declares that a User has ChatRoom as its navigation parent. This relationship is independent of inheritance: a PremiumUser might inherit commands from User while having ChatRoom as its navigation parent.

Programmatic Parent Resolution. Each context provides a method that returns the actual parent object for a given context instance. For a User object, this method returns the specific ChatRoom instance that contains this user. This programmatic resolution enables navigation based on runtime object relationships rather than static configuration.

Bidirectional Navigation. The framework supports navigation in both directions:

- **Downward navigation** occurs when a command changes the current context to a child object. For example, a command which sets the current user in ChatRoom updates the current command context, changing the active context from ChatRoom to User.
- **Upward navigation** occurs via a built-in command that retrieves the parent object via a method and sets it as the

current context. Users can invoke this with natural language like “go back to chatroom” to return to the parent context.

This bidirectional navigation mirrors how users naturally think about hierarchical applications, drilling down to specific objects and backing out to broader contexts.

A.6.3 Command Resolution and Action Space Reduction. At workflow load time, the system computes the effective command set F_c for each context c by recursively resolving the inheritance graph. The resolution algorithm performs depth-first traversal: for each base context, its effective command set is computed recursively, then merged with the current context’s own commands (own commands taking precedence on name collisions).

This pre-computation means the agent always operates over a flattened, pre-resolved command set—no inheritance traversal occurs at runtime. The system caches the resolved command sets for each context, ensuring $O(1)$ lookup when determining which commands are available in the current context.

The result is that each context exposes precisely the commands relevant to its domain, reducing $|F_c|$ from potentially hundreds of commands (in a flat namespace) to tens of contextually appropriate commands. This targeted reduction directly addresses the tool overload failure mode (Section 3.2.1).

Upward Command Search. When a command cannot be resolved in the current context’s effective command set, the system traverses upward through the navigation hierarchy, checking each ancestor context’s command set. This enables graceful degradation: a user working in a deeply nested context (e.g., OrderItem) can still invoke commands from broader parent contexts (e.g., Order, Customer) without explicit context switching. The search terminates when a matching command is found or the root context is reached.

A.7 Adaptive Context Management Details

Command Catalog Separation. Command schemas and dependency hints are injected into the system message via a custom prompt adapter rather than included in the trajectory. This prevents the agent from “forgetting” available commands as older trajectory entries are truncated, a failure mode that leads to schema hallucination and capability blindness in naive implementations.

Trajectory Truncation. When token count approaches the context limit, the system truncates in units of complete interaction cycles (thought, tool name, tool args, observation), preserving the original user query and the most recent k cycles to maintain task coherence.

State Externalization. Critical workflow state with partially extracted parameters (when present), NLU pipeline stage, and recent action history is maintained in persistent external storage rather than reconstructed by reasoning over trajectory history, converting state tracking from a memory-intensive reasoning task into a simple lookup.

Conversation History Summarization. At the end of each turn, an LLM-based summarizer generates a compact summary capturing relevant facts and parameter values while omitting procedural details. These summaries are injected at the start of subsequent turns, providing multi-turn context in a few hundred tokens without maintaining the full conversation trajectory.

A.8 Insights Distillation

A fundamental challenge in deploying small models for agentic tasks is the reasoning quality gap: small models systematically make mistakes that larger models avoid, but fine-tuning small models requires expensive labeled data. fastWorkflow addresses this through insights distillation, a teacher-student comparison mechanism that automatically extracts actionable insights from divergent behavior and injects them into future prompts. Figure 8 illustrates this distillation architecture.

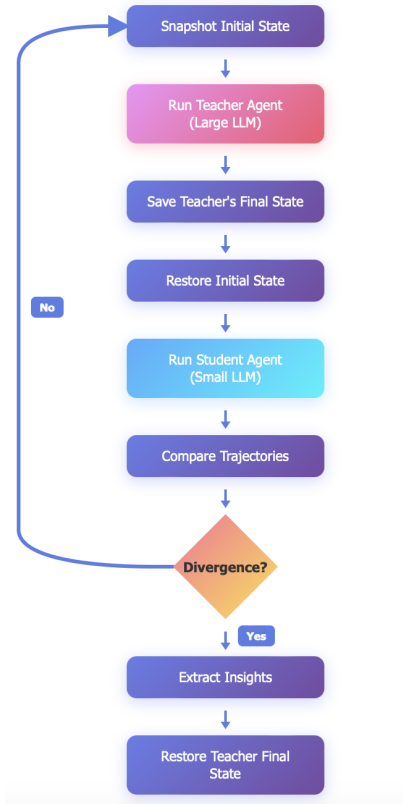


Figure 8: Insights Distillation Architecture via Teacher-Student Comparison. The system snapshots the initial workflow state, runs a capable Teacher Agent (Large LLM), and saves its final state. The state is then restored so the Student Agent (Small LLM) can process the same query under identical conditions. Trajectories are compared for divergence. If divergent, the system extracts actionable insights and restores the Teacher’s authoritative final state. If no divergence is detected, the workflow continues without interruption.

A.8.1 Teacher-Student Comparison Framework. The distillation process operates by running both a “teacher” (large, capable LLM) and a “student” (small, deployable LLM) on identical user queries and comparing their behavior at two levels:

Planning-Level Comparison. Both agents invoke the task planner (Section 4.3) before execution. The distillation system captures each planner’s generated step sequence and compares them. Differences

in plan structure: missing steps, incorrect ordering, omitted dependencies indicate planning-level reasoning gaps that the student exhibits.

Execution-Level Comparison. As each agent executes its plan, the system records the actual commands invoked along with their resolved parameters. These action records provide ground truth about what each agent actually did, independent of what it planned. Execution-level divergences reveal mistakes in tool selection, parameter formatting, or plan interpretation.

The comparison operates on action signatures (command name, parameters) tuples, enabling detection of three divergence types: actions executed only by the teacher (student omitted necessary steps), actions executed only by the student (student performed unnecessary or incorrect steps), and ordering differences (student executed correct actions in wrong sequence).

Before running the teacher, the system captures the complete workflow state: the workflow context dictionary, the CME (command metadata extraction) workflow state, and the conversation history. The teacher agent runs to completion, producing its trajectory, actions, and plans. The teacher’s final state is captured as a separate snapshot. The system restores the pre-execution snapshot, returning the workflow to its original state. The student agent runs on the restored state, experiencing identical initial conditions to the teacher. If divergence is detected (insights were extracted), the teacher’s final state is restored as the authoritative outcome. If no divergence occurred, the student’s state is retained (equivalent to the teacher’s).

A.8.2 Insight Extraction. When divergences are detected, the system invokes a dedicated insight extraction module to analyze the differences and generate actionable rules. The extraction process differs for planning versus execution insights:

Planning Insight Extraction. The extraction signature receives: the user’s original query, the teacher’s plan (correct approach), the student’s plan (potentially flawed), a summary of how the plans diverged, the actual executed actions from both agents and existing insights (for deduplication). The extraction module is instructed to generate prescriptive rules (“what to do”). The module returns EMPTY if differences are context-justified or would duplicate existing insights.

Execution Insight Extraction. The extraction signature receives: the user’s query, the full workflow agent trajectories from both agents (including all thoughts, tool calls, arguments, and observations), a summary of action-level differences and existing insights (for deduplication). The full trajectories provide context for understanding why each agent made its choices. The extraction module generates anti-patterns (“what not to do”). These negative rules help small models avoid specific mistake patterns observed during distillation.

A.8.3 Multi-Turn Context Awareness. A subtle challenge arises in multi-turn conversations: teacher and student may have legitimately different behavior due to different conversation history context from prior turns. A difference that appears to be a mistake may actually be a reasonable response to different information.

The insights extraction signatures explicitly address this through context-divergence awareness. Before extracting a rule, the LLM is instructed to analyze:

- (1) Whether the planning/execution difference was a genuine mistake (extract a rule)
- (2) Whether the difference was justified by different context (return EMPTY)

This filtering prevents the system from generating spurious insights based on context-justified differences, ensuring the extracted rules reflect genuine reasoning gaps.

A.8.4 Insight Persistence and Injection. Extracted insights are persisted to workflow-specific markdown files in a designated Insights directory. Each insight is numbered and appended to the appropriate file. At runtime, these insights are loaded and injected into agents system prompts.

The distillation mechanism can be viewed as a form of offline training that improves small model performance without fine-tuning. Once sufficient insights have accumulated, the system can run in student-only mode with insight injection. The small model benefits from the accumulated reasoning patterns without requiring the teacher model at runtime.

This enables cost-effective deployment: expensive teacher models are used during training to generate insights, while cheap student models handle production traffic with injected insights closing much of the reasoning gap.

Mitigation Coverage. Insights distillation addresses the fundamental reasoning quality gap between small and large models by encoding successful reasoning patterns directly into prompts. Planning insights address decomposition failures by providing prescriptive rules for task breakdown. Execution anti-patterns address tool misuse and parameter errors by explicitly warning against observed mistake patterns. The teacher-student comparison framework enables automatic insight extraction without manual labeling.

A.9 Training Phase Details

fastWorkflow generates synthetic training data automatically from example utterances defined for each command. The volume of generated data is controlled by configurable environment variables, specifically, the number of personas, the number of utterances per persona, and the number of personas per batch. This makes the data generation process both flexible and domain-agnostic, requiring minimal human effort to adapt to new workflows.

Training: The two classifiers in the cascaded NLU pipeline, DistilBERT and BERT-Tiny are then trained on this synthetically generated data. Training time naturally scales with the data generation parameters, but as demonstrated on the Banking77 benchmark (10,000+ samples, 77 intent classes), both models are lightweight and train within minutes on standard hardware: DistilBERT at approximately 41–43 seconds per epoch and BERT-Tiny at 10–12 seconds per epoch.

Parameter Extraction Few-Shot Examples: In addition to intent classification, fastWorkflow trains a parameter extraction component which generates synthetic labeled few-shot examples for the parameter extractor. These examples (configurable via a hyperparameter, we set it to 15 for our workflows) are generated once per

workflow, per command and cached, taking approximately 30 seconds for the 15 commands in retail workflow (7.8% of total training time).

Overall, the training phase is designed to be low-overhead and easily adaptable. Users can tune the data generation variables to match their domain’s complexity, and the full pipeline including intent classification training and parameter extraction example generation can be retrained quickly without requiring large-scale compute resources. On the retail workflow, end-to-end training (including one-time internal setup) completes in under 7 minutes (~397 seconds) on standard hardware.

B Extended Results

B.1 Intent Detection Study

We validate the two-stage classifier cascade (Section 4.1.1) through a controlled evaluation on Banking77 [4], a challenging intent classification benchmark with 77 fine-grained intents across 10,003 training samples. We repartition the original test set into 2,464 test and 616 validation samples. We report both F1 score and NDCG@3 (normalized discounted cumulative gain at rank 3), the latter capturing ranking quality of the top- k predictions used for ambiguity clarification.

B.1.1 Model Selection. We fine-tune five sentence transformer architectures as candidate classifiers and evaluate them on accuracy, training cost, and memory footprint (Table 3).

Table 3: Classifier model comparison on Banking77. Training time is for 3 epochs on a T4 GPU.

Model	F1	NDCG@3	Train Time	GPU Mem
ModernBERT	0.924	0.956	350s	4.6 GB
DeBERTa	0.919	0.952	332s	10.4 GB
DistilBERT	0.910	0.951	124s	2.3 GB
all-MiniLM-L6-v2	0.809	0.899	65s	0.9 GB
BERT-Tiny	0.758	0.865	31s	0.3 GB

While ModernBERT achieves the highest accuracy, it requires $11\times$ longer training and $15\times$ more memory than BERT-Tiny. For the two-stage cascade, we select **BERT-Tiny** (4.4M parameters, 17.6 MB) as the Stage 1 classifier and **DistilBERT** (66M parameters, 267.8 MB) as the Stage 2 escalation model. This pairing optimizes for the cost-accuracy tradeoff described in Section 4.1.1: BERT-Tiny provides fast inference for high-confidence predictions while DistilBERT handles uncertain cases with near-best accuracy (F1 within 1.4 points of ModernBERT) at moderate resource cost.

B.1.2 Escalation Threshold Optimization. The escalation threshold $\tau_{escalation}$ determines when BERT-Tiny defers to DistilBERT. We optimize $\tau_{escalation}$ by maximizing:

$$F1 \times NDCG@3 \times (1 - \text{DistilBERT Usage})$$

balancing accuracy against computational cost. The optimal threshold is $\tau_{escalation} = 0.533$, at which the cascade achieves $F1 = 0.917$ and $NDCG@3 = 0.951$ while routing only 26.79% of utterances to DistilBERT. This means BERT-Tiny confidently handles approximately three-quarters of inputs, reducing average inference cost

while improving accuracy by about 0.8% relative to using DistilBERT alone for all predictions.

B.1.3 Pipeline vs. Semantic Router Baseline. We compare the two-stage classifier cascade against a semantic router baseline (cosine similarity over sentence embeddings using all-MiniLM-L6-v2) on Banking77 (Table 4).

Table 4: Two-stage classifier cascade vs. semantic router on Banking77.

Metric	Semantic Router	Cascade Pipeline	Relative Gain
Validation F1	0.906	0.923	+1.88%
Validation NDCG@3	0.913	0.949	+3.95%
Validation Time	7.0s	<0.5s	>14× faster
Test Time	38.0s	1.0s	38× faster

The cascade pipeline improves ranking quality (NDCG@3) by +3.95% over the semantic router while achieving 38× faster test-time inference. The NDCG@3 improvement is particularly important for the ambiguity clarification mechanism (Section 4.1.1): when the classifier’s confidence falls below $\tau_{ambiguity}$, the top- k predictions are presented to the user for disambiguation, making ranking quality directly impact user experience. The inference speedup stems from replacing exhaustive embedding similarity computation over all intents with a lightweight forward pass through BERT-Tiny, escalating to DistilBERT only when necessary.

B.1.4 Embedding Quality. PCA visualization of BERT-Tiny embeddings before and after fine-tuning confirms that the training procedure produces well-separated intent clusters. Quantitatively, the average centroid distance between intent clusters increases by 304.9% after fine-tuning, while the separation ratio (inter-class distance / intra-class distance) improves by 263.1% (from 0.879 to 3.191). This validates that even ultra-lightweight models can learn discriminative intent representations when fine-tuned on domain-specific synthetic utterances, supporting the cascade design where BERT-Tiny serves as an effective first-pass filter.

B.2 Parameter Extraction Study

We validate the extraction pipeline design through a controlled evaluation on a subset derived from CLINC_OOS [15]. From its 150-intent corpus, we select 15 parameter-rich intents spanning diverse domains (restaurant reservation, flight booking, meeting scheduling, weather, transfers, etc.) with heterogeneous parameter types (strings, integers, floats, booleans, enums). We evaluate 300 labeled examples using a F1 score that penalizes both hallucinated values (lowering precision) and missing values (lowering recall).

B.2.1 Baseline Performance and Scaling. Table 5 shows parameter extraction F1 scores across model scales.

The largest stepwise gain (+0.237) occurs when scaling from 0.5B to 24B parameters, while further scaling to 40B yields only marginal improvement (+0.027), indicating diminishing returns beyond the mid-scale regime.

Table 5: Baseline parameter extraction F1 scores by model scale.

Model	Params	F1
Qwen-2.5-0.5B-Instruct	0.49B	0.568
Mistral Small 3.1	24B	0.805
Gemini 2.0 Flash	40B	0.832

B.2.2 Extraction Strategy and Optimizer Selection. We select ChainOfThought over dspy.Predict module because it substantially reduces hallucinations in extracting parameters. For optimizer selection, we compare four DSPy teleprompters (Table 6). LabeledFewShot achieves F1 within 0.6% of the best performer (BootstrapFewShotWithRandomSearch) at millisecond (fastest) optimization time, making it the practical choice for a framework requiring rapid reconfiguration across workflow commands.

Table 6: DSPy optimizer comparison on parameter extraction.

Optimizer	F1
BootstrapFewShotWithRandomSearch	0.995
MIPROv2	0.991
LabeledFewShot	0.989
BootstrapFewShot	0.980

B.2.3 Optimization Impact. Applying MIPROv2 to Mistral Small 3.1 and Gemini 2.0 Flash yields average F1 improvements of +7.6% and +5.75% respectively across all 15 domains, with peak gains of +23.3% (Transfer) and +17.0% (How Busy). Notably, despite lower baseline performance, Mistral Small 3.1 achieves comparable post-optimization F1 (0.952 vs. 0.950) and leads in more domains (6 vs 3), while offering 2× lower latency and lower output token cost, making it the preferred extraction model.

B.2.4 Error Analysis. We categorize baseline extraction errors (130 for Mistral, 88 for Gemini across 300 queries) into three classes (Table 7).

Table 7: Parameter extraction error distribution by category.

Error Category	Mistral	Gemini
Format Inconsistencies	41	33
Hallucinations	39	28
Wrong Extraction	50	27
Total	130	88

The dominant error pattern across both models is *default value hallucination* (25–30% of errors): predicting plausible default values for parameters absent from the query rather than returning a sentinel. Format inconsistencies (date/time variations, currency notation) justify the multi-layer validation cascade, which resolves these deterministically without re-invoking the LLM.

B.3 Cost Analysis

We conducted a detailed per-component cost analysis on the τ -bench Retail domain dev set tasks for both GPT-OSS-20B (via Groq) and Claude Opus 4.1 (via Anthropic API). Each part of fastWorkflow (planner, agent, param_extraction, summarizer) is instrumented with a label before execution. When the LLM responds, the API reports exactly how many tokens were in the prompt and how many were generated. We accumulate these per label and multiply token counts by the published per-token price for the model used, tracking input and output tokens separately as they have different pricing.

B.3.1 Inference Cost Breakdown. Tables 8 and 9 report the per-component cost breakdown for GPT-OSS-20B and Claude Opus 4.1 respectively.

Table 8: Per-component inference cost for GPT-OSS-20B + fastWorkflow on τ -bench Retail (20 tasks). Groq pricing: \$0.075/M input, \$0.30/M output.

Component	Input \$	Output \$	Total \$	\$/Task	%
Agent	0.1447	0.0928	0.2375	0.0119	70.3
Param. Extr.	0.0018	0.0029	0.0047	0.0002	1.4
Planner	0.0416	0.0379	0.0794	0.0040	23.5
Summarizer	0.0067	0.0094	0.0161	0.0008	4.8
Total	0.1948	0.1429	0.3377	0.0169	100.0

Table 9: Per-component inference cost for Claude Opus 4.1 + fastWorkflow on τ -bench Retail (20 tasks). Anthropic pricing: \$5.0/M input, \$75.0/M output.

Component	Input \$	Output \$	Total \$	\$/Task	%
Agent	9.2570	6.9758	16.2328	0.8116	69.3
Param. Extr.	0.2000	0.4498	0.6497	0.0325	2.8
Planner	2.1034	2.6059	4.7094	0.2355	20.1
Summarizer	0.5456	1.2971	1.8427	0.0921	7.9
Total	12.1060	11.3286	23.4346	1.1717	100.0

The cost ratio is stark: Claude Opus 4.1 costs approximately 69 $\times$ more per task than GPT-OSS-20B under the same fastWorkflow framework (\$1.17/task vs. \$0.017/task). Yet GPT-OSS-20B + fastWorkflow achieves 83.47% Pass¹ on Retail—comparable to Opus 4.1’s leaderboard score of 82.40% without fastWorkflow. This means GPT-OSS-20B with fastWorkflow delivers frontier-level performance at less than 1.5% of the cost of running the frontier model itself.

The component breakdown also reveals that the fastWorkflow overhead (planner, summarizer, parameter extraction) is modest relative to the agent’s own inference: these components collectively account for $\sim$ 30% of total cost, with parameter extraction contributing only $\sim$ 1.4%. The BERT-Tiny and DistilBERT classifiers used for intent detection add sub-millisecond latency and negligible compute cost (not reflected in the LLM cost tables above as they run locally).

B.3.2 Training Cost. Training incurs two types of external API cost, both using LiteLLM. First, synthetic utterance generation makes $\sim$ 4 API calls per command (one per persona batch) to generate 5 diverse utterances each, roughly 60 calls total for a 15-command workflow like Retail. Second, DSPy few-shot example generation makes $\sim$ 1 API call per command with parameters to generate 15 labeled examples, adding $\sim$ 15 more calls. All model training (DistilBERT, BERT-Tiny) runs entirely locally with no API cost. The full pipeline completes in under 7 minutes on standard CPU hardware.

B.3.3 Baseline Cost Comparison. To enable a direct cost comparison, we also measured the inference cost of Claude Opus 4.1 running the standard τ -bench leaderboard configuration (a multi-turn tool-calling agent) on the same 20 Retail tasks (Table 10).

Table 10: Cost comparison: Claude Opus 4.1 without fastWorkflow vs. GPT-OSS-20B + fastWorkflow on τ -bench Retail Dev Set.

Configuration	Input \$	Output \$	Total \$	\$/Task	Pass ¹
Opus 4.1 (w/o fastWorkflow)	5.5809	1.9103	7.4912	0.3746	82.4%
GPT-OSS-20B + fastWorkflow	0.1948	0.1429	0.3377	0.0169	83.47%

Without fastWorkflow, Claude Opus 4.1 costs \$0.375/task and achieves 82.4% Pass¹. OSS 20B with fastWorkflow costs \$0.017/task and achieves 83.47% Pass¹ delivering comparable performance at $\sim$ 22 $\times$ lower cost. This is the basis for the cost comparison reported in the main paper. Notably, the baseline Opus 4.1 configuration uses fewer tokens per task on average (55,809 input, 1,274 output) than GPT-OSS-20B + fastWorkflow (129,835 input, 23,825 output). The cost advantage of GPT-OSS-20B is therefore driven entirely by the $\sim$ 67 $\times$ difference in per-token pricing, which more than compensates for the higher token consumption introduced by fastWorkflow’s multi-component architecture. Note that the $\sim$ 22 $\times$ cost advantage refers to per-task inference cost only and does not include the one-time training phase cost (classifier fine-tuning and synthetic data generation), which is amortized across all subsequent tasks. Also, the cost estimates above use Groq pricing (\$0.075/M input, \$0.30/M output) for GPT-OSS-20B. Alternative providers offer lower rates: for example, OpenRouter prices GPT-OSS-20B at \$0.03/M input and \$0.14/M output, which would reduce the per-task cost from \$0.017 to approximately \$0.007, widening the cost advantage to $\sim$ 52 $\times$ over the Opus 4.1 baseline.

B.4 Multiple Trial Results

To quantify variance from LLM sampling, we report Pass^k results for GPT-OSS-20B + fastWorkflow across three independent trials on both τ -bench domains (Table 11).

The drop from Pass¹ to Pass³ on both domains (83.47% $\rightarrow$ 53.00% on Retail, 78% $\rightarrow$ 52% on Airline) indicates that while the system succeeds on most tasks at least once, a subset of tasks exhibit non-trivial variance across runs. This is consistent with the stochastic nature of LLM generation: tasks at the boundary of model capability may succeed or fail depending on sampling outcomes. Notably, the convergence of Pass³ scores across both domains ($\sim$ 52–53%) suggests a shared floor of reliably solvable tasks,

Table 11: Pass^k results for GPT-OSS-20B + fastWorkflow on τ -bench. Pass^k measures the fraction of tasks where *all* k independent trials succeed, capturing consistency under LLM sampling variance.

Metric	Retail	Airline
Pass ¹	83.47%	78%
Pass ²	71.40%	67%
Pass ³	53.00%	52%

with the remaining $\sim 25\text{--}30\%$ of tasks being sensitive to sampling variance.

B.5 Evaluation Traces and Reproducibility Artifacts

To support reproducibility, we release a curated set of evaluation traces from our τ -bench and τ^2 -bench experiments. The repository includes per-task agent trajectories for fastWorkflow-augmented runs across all evaluated models (Mistral-7B-Instruct, Qwen3-14B, GPT-OSS-20B) and some other models too. The artifacts are available at: <https://github.com/sanchit056/tau-bench-traces-examples-and-proofs>.